ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Group Project: Foundations of Computational and Typing Systems

Overview

This second project consists of five exercises. Exercises 1 and 2 focus on computations in the lambda calculus. The second part, with exercise 3 is on inductive definitions. The third part, with Exercises 4 and 5 are on typing.

Groups

- Form groups of *3*.
- **Constraint**: no two students who were in the same group for Project 1 may be in the same group for Project 2. Groups that violate this rule will be asked to reshuffle.

Deliverables

- Exercises 1 and 2 require the the Lambda Calculus Interpreter available on Moodle. Write your code in a file GroupID_Project_2.lc
- Exercises 3-5 are pen-and-paper. Write your solution in markdown (or an alternative typesetting format of your choice)

Oral presentation

10-15 minutes per group (including questions)

Exercise 1 — Programming in the Lambda Calculus

Using the Church encodings from the lectures (booleans, numerals, pairs, lists) and the interpreter, implement the following.

(a) Comparisons. Define lambda terms for the following comparison operators on Church numerals:

- **leq** (less than or equal): leq m n returns true if m ≤ n, false otherwise
- **lt** (strictly less than): lt m n returns true if m < n

You will need a predecessor or subtraction operator as an auxiliary — define that too. Test your definitions in the interpreter on at least three representative cases each (including an edge case with 0).

(b) Integer division. Define a lambda term **div** that computes integer division (quotient) on Church numerals: div m n should return ⌊m/n⌋ for n ≥ 1.

Hint: think recursively (in a call by name setting) — count how many times you can subtract n from m before reaching 0.

Test div 6 3, div 7 2, and div 2 5 in the interpreter.

(c) Filter. Consider the following encoding of lists:

```
nil  = λc. λn. n
cons = λh. λt. λc. λn. c h (t c n)
```

A list is a function that takes a "combining function" `c` and a "base value" `n` and folds over its elements. For example:

```
[]        = nil                          = λc. λn. n
[1]       = cons 1 nil                   = λc. λn. c 1 n
[1, 2]    = cons 1 (cons 2 nil)          = λc. λn. c 1 (c 2 n)
[1, 2, 3] = cons 1 (cons 2 (cons 3 nil)) = λc. λn. c 1 (c 2 (c 3 n))
```

Notice that in the definition of cons, the parameter t is the tail of the list — itself a Church-encoded list, and therefore a function. The expression `tcn` applies the tail to the same combining function and base value, effectively folding over the rest of the list. The recursive traversal is not achieved through an explicit recursive call, but through the structure of the encoding itself: each tail already "knows" how to fold over its own elements.

Define a lambda term **filter** that takes a predicate (a function returning a Church boolean) and a list, and returns the list of elements satisfying the predicate.

Test it by filtering the list [1, 2, 3, 4, 5, 6] with the predicate that keeps only the elements greater than 3 (using your **lt** from part (a)).

Exercise 2 — Call-by-Name vs Call-by-Value

The interpreter supports both call-by-name and call-by-value evaluation. Use it to explore the differences.

(a) Divergence. Divergence through self-application. Define the following terms:

$$\mathsf{A} = \lambda \mathsf{f}.\lambda \mathsf{x}.\mathsf{f}\,(\mathsf{f}\,\mathsf{x})$$
$$\mathsf{B} = \lambda \mathsf{x}.\mathsf{x}$$
$$\mathsf{W} = \lambda \mathsf{x}.\mathsf{x}\,\mathsf{x}\,\mathsf{x}$$

- Evaluate $\mathsf{AB}(\mathsf{WW})$ under both call-by-name and call-by-value. One strategy terminates and one diverges — determine which, show the evaluations in the interpreter, and explain why.
- Now swap the argument order: evaluate $A(WW)B$ under both strategies. Does the outcome change compared to (a)? Explain in terms of which rules fire and in which order.

(b) Wasted work. Define a term G such that under call-by-value, the argument is evaluated once, while under call-by-name, it is evaluated multiple times.

Show both evaluations in the interpreter (counting the steps).

(c) Short-circuit evaluation. Recall the Church encoding of the conditional. Explain why the Church-encoded conditional does **not** short-circuit under call-by-value: that is, both branches are evaluated regardless of the guard.

Define a term **ifc** (a short-circuiting conditional) that, under call-by-value, evaluates only the selected branch. Demonstrate the difference by applying both the standard conditional and your ifc to a case where the non-selected branch diverges.

Exercise 3 — Inductive Definitions and Rule Induction

Consider the following inductive definition of the judgment **s balanced**, which characterizes strings of balanced parentheses:

$$\frac{}{\varepsilon \text{ balanced}} \quad (\text{bal-empty}) \qquad \frac{s \text{ balanced}}{(\ s\ ) \text{ balanced}} \quad (\text{bal-paren}) \qquad \frac{s_1 \text{ balanced} \quad s_2 \text{ balanced}}{s_1\, s_2 \text{ balanced}} \quad (\text{bal-concat})$$

where ε denotes the empty string and juxtaposition denotes string concatenation.

(a) Construct derivation trees for: (()) balanced and ()(()) balanced.

(b) State the principle of rule induction for the judgment s balanced. That is: to prove that a property P(s) holds for all s such that s balanced, what must you show?

(c) Define the *nesting depth* of a string of parentheses as follows:

- depth(ε) = 0
- depth((s)) = 1 + depth(s)
- depth(s_1 s_2) = max(depth(s_1), depth(s_2))

and define len(s) as the total number of parenthesis characters (both open and close) in s.

Using rule induction, prove that for all balanced strings s:

$$\text{len}(s) \geq 2 \cdot \text{depth}(s)$$

(d) Is the string)(balanced? Either give a derivation, or prove that no derivation exists.

Hint: find a property that holds of all balanced strings (provable by rule induction) but fails for)(.

Exercise 4 — Typing Derivations

Consider the simply typed lambda calculus with Nat, Bool, +, =, if-then-else, and fix, as defined in Lecture 9.

(a) Construct a complete typing derivation (as a proof tree) for the following term:

```
⊢ (λf : Nat → Nat. λx : Nat. f (f x)) (λy : Nat. y + 1) : Nat → Nat
```

Write out every rule application explicitly, labelling each node with the rule name.

(b) Consider the term:

```
⊢ fix@(Nat → Nat). (λf : Nat → Nat. λn : Nat. if (n = 0) then 0 else (f (n + 1)) + n)
```

Construct its typing derivation. Then reduce the term applied to 2 for three reduction steps (using the reduction rules from Lecture 9) and verify explicitly that the type is preserved at each step (i.e., re-derive the type of each intermediate term).

(c) Show that the following term is **ill-typed** by identifying the exact point in the bottom-up type-checking algorithm where a mismatch occurs:

```
λf : Nat → Bool. λx : Nat. (f x) + 1
```

State precisely which rule is being applied, what types are expected, and what type is actually found.

Exercise 5 — Type Inference

Consider the implicitly typed lambda calculus (i.e., abstractions are written λx. M without type annotations) with the constraint-generation algorithm from Lecture 9.

(a) Apply the inference algorithm to the term:

```
λf. λg. λx. if (f x) then (g x) else x
```

Write down the full trace: for each subterm, state the fresh type variables introduced, the recursive calls, and the constraints generated. Collect all constraints and solve them by hand (applying unification). State the principal type.

(b) Apply the inference algorithm to the term:

```
λf. f (f 1) (f true)
```

Generate the constraints and show that they have **no solution**. Identify precisely which pair of constraints is contradictory and explain in plain language what the contradiction means — what would the type of f need to be, and why is that impossible in the simply typed system?

Good luck — and have fun!